Databases – Lecture Notes: ER to Relational Model

Yonghao Lee

March 23, 2026

Contents

1 From ER Diagrams to the Relational Model **2**
1.1 The Relational Data Model 2
1.2 Translating Entity Sets 2
1.3 Translating Relationship Sets (Without Constraints) 3
1.4 Self-Referencing (Recursive) Relationships 3
1.5 Relationships with Constraints (One-to-Many) 4
1.6 Ternary Relationships with Constraints 5
1.7 Participation Constraints (Exactly One) 5
1.8 Ternary Relationships with Mandatory Constraints 6
1.9 Translating Weak Entity Sets 6
1.10 Translating ISA (Inheritance) Relationships 7

2 Comprehensive Example: Schema Translation **9**

---

1 From ER Diagrams to the Relational Model

The database design process follows three fundamental phases:

1. **Conceptual Design:** Constructing Entity-Relationship (ER) diagrams.
2. **Logical Design:** Translating the ER diagram into a **Relational Model** (a structured collection of tables and constraints).
3. **Physical Design:** Utilizing Data Definition Language (DDL), such as SQL, to implement the schema within a Database Management System (DBMS).

This section focuses strictly on Logical Design: the systematic translation of visual ER diagrams into formal relational schemas.

1.1 The Relational Data Model

The relational model is grounded in mathematical set theory, conceptualized structurally as a collection of two-dimensional **tables**.

Definition 1.1 Relational Concepts

- **Relation:** A table structure.
- **Tuple:** A single row representing a discrete record.
- **Instance:** The collection of tuples currently populated within the table.
- **Schema:** The structural blueprint defining the relation's name and its constituent attributes (columns), e.g., `Movies(title, year, length, genre)`.

Keys in the Relational Model

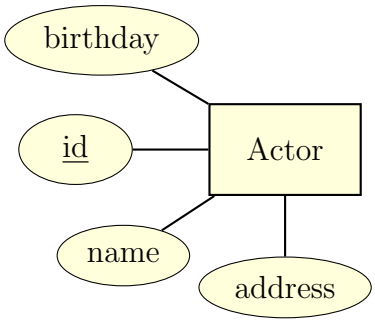
Every relation requires a mechanism to uniquely identify each tuple. A **key** is a minimal set of attributes guaranteeing uniqueness. Primary keys are denoted via <u>underlining</u>.

- `Student(`<u>`id`</u>`, name, address)`
  Key: `id`. Ensures uniqueness despite shared names or addresses.
- `Studies(`<u>`studentId`</u>`, `<u>`courseId`</u>`)`
  Key: The composite of `(studentId, courseId)`. Required because neither attribute guarantees uniqueness independently in a many-to-many relationship.
- `FinalGrade(`<u>`studentId`</u>`, `<u>`courseId`</u>`, grade)`
  Key: `(studentId, courseId)`. The `grade` is excluded from the key because a student receives exactly one final grade per course; the IDs are sufficient for identification.

1.2 Translating Entity Sets

Translation Rule: Entity Set

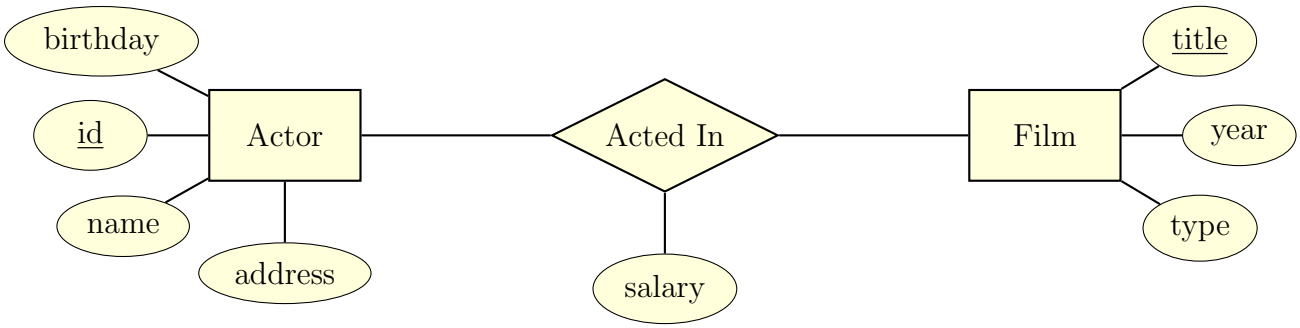
Construct a relation corresponding to the entity set's name. The attributes of the relation map directly to the entity's attributes. The ER diagram's primary key remains the relation's primary key.



Resulting Schema: `Actor(id, name, birthday, address)`

1.3 Translating Relationship Sets (Without Constraints)

Unconstrained relationships (denoted by unadorned connecting lines) represent **many-to-many** mappings. These require a distinct associative relation to avoid data anomalies.



Translation Rule: Unconstrained Relationship

Define a new relation encompassing:

1. The **primary keys** of all participating entities (acting as foreign keys).
2. Any descriptive attributes attached directly to the relationship (e.g., `salary`).

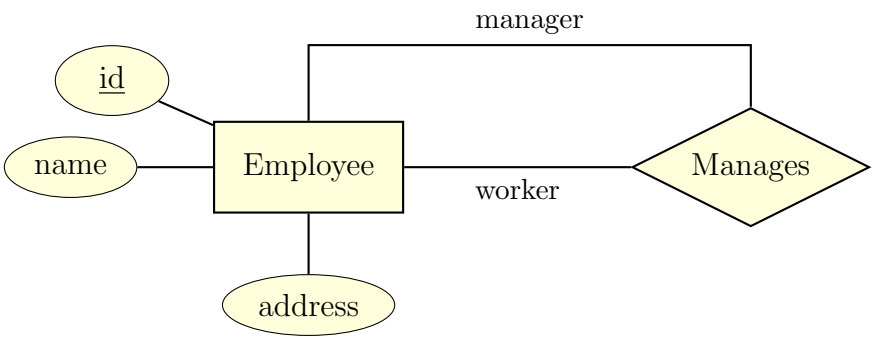
The primary key of this associative relation is the **composite** of the participating entities' primary keys.

Resulting Schema:

- `Actor(id, name, birthday, address)`
- `Film(title, year, type)`
- `ActedIn(actor_id, film_title, salary)`

1.4 Self-Referencing (Recursive) Relationships

Recursive relationships connect an entity set to itself (e.g., organizational hierarchies).



The Role Renaming Rule

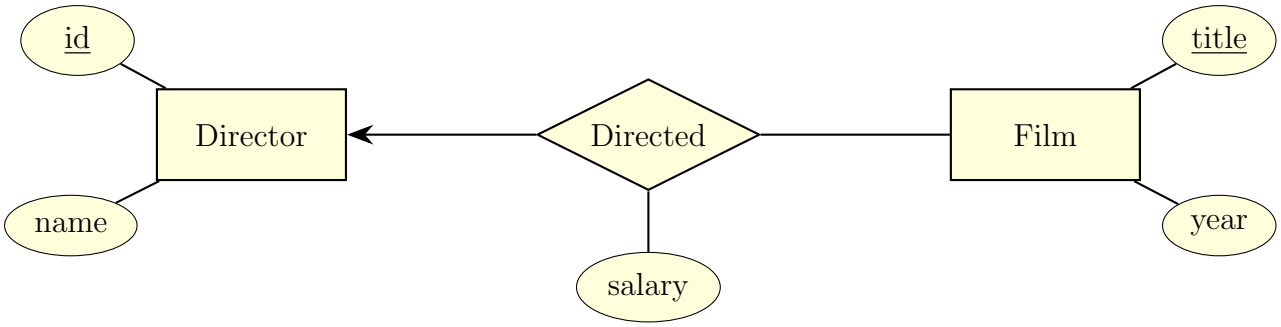
Applying standard translation would yield `Manages(id, id)`, resulting in duplicate attribute names and semantic ambiguity.
Resolution: Utilize the **role indicators** assigned to the ER diagram edges (e.g., "manager" and "worker") to rename the respective foreign keys, ensuring distinction.

Resulting Schema:

- `Employee(id, name, address)`
- `Manages(managerId, workerId)`

1.5 Relationships with Constraints (One-to-Many)

A pointed arrow (→) indicates a **one-to-many** (or many-to-one) cardinality.



The One-to-Many Translation Rule

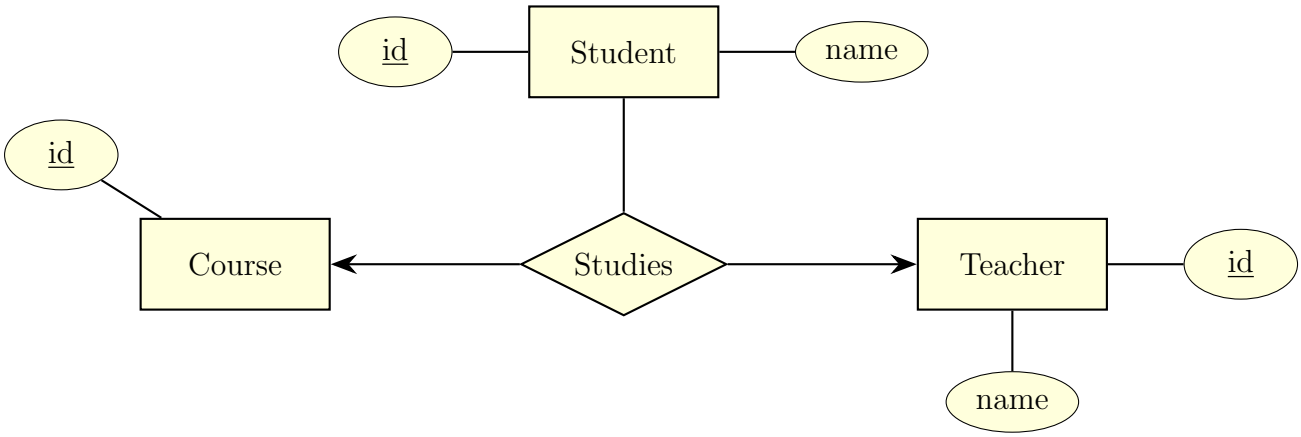
The arrow establishes that **a Film is directed by at most one Director**.
When formulating an associative relation, the primary key is derived **exclusively** from the entity on the unconstrained ("many") side (the plain line).

Resulting Schema: The `id` from `Director` acts purely as a foreign key and is excluded from the primary key definition.

- `Director(id, name)`
- `Film(title, year)`
- `Directed(title, id, salary)`

1.6 Ternary Relationships with Constraints

The logic for binary relationships extends natively to n-ary relationships. Unconstrained edges (plain lines) dictate the primary key formulation.



Translating n-ary Constraints

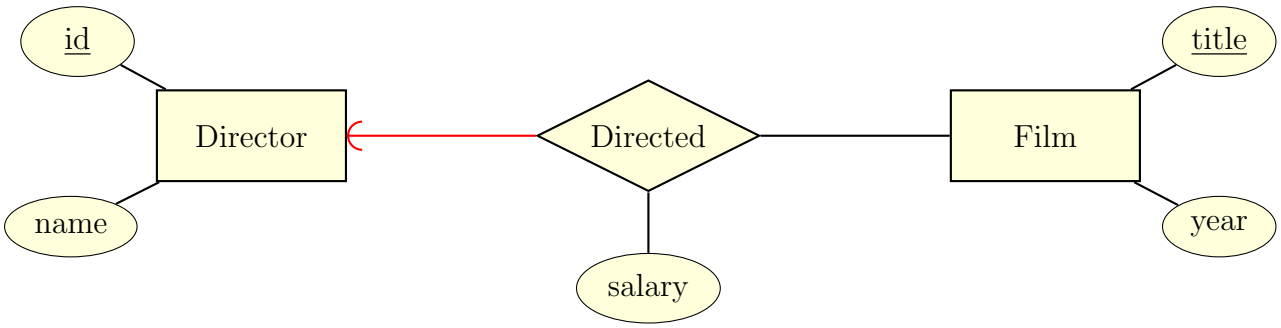
Arrows indicate that *a specific Student can be associated with at most one Course and at most one Teacher*. The student functionally determines the other participants.
The Rule: The primary key of the associative relation is constituted solely by the primary key(s) of the entity connected via a **plain line**.

Resulting Schema:

- `Course(id)`
- `Student(id, name)`
- `Teacher(id, name)`
- `Studies(cid, sid, tid)`

1.7 Participation Constraints (Exactly One)

A rounded arrow imposes an "exactly one" cardinality constraint, signifying **mandatory participation** and strict functional dependence.



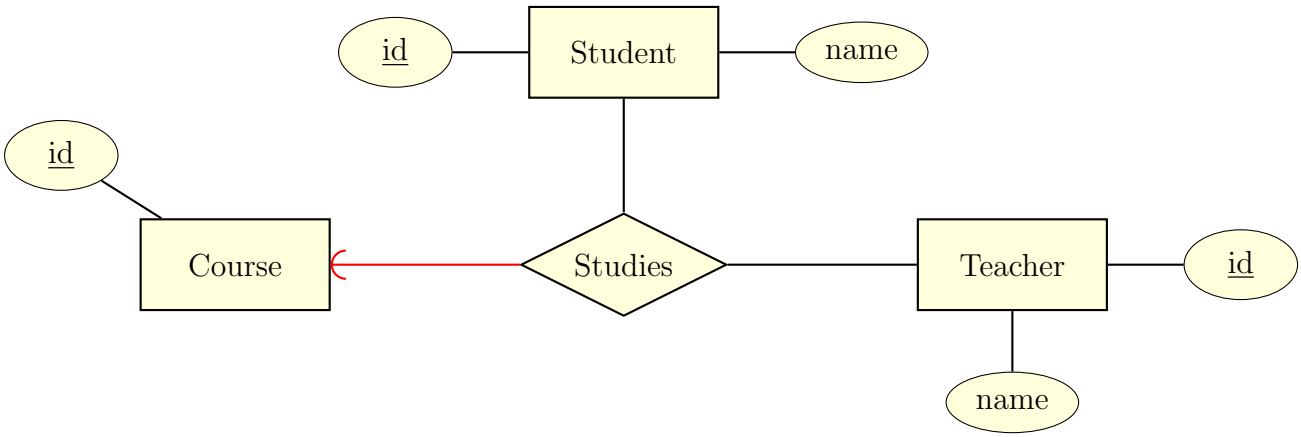
The "Exactly One" Optimization

Because participation is mandated, instantiating an independent `Directed` relation introduces structural inefficiency.
Optimization: Do **not** instantiate a separate relation. Subsume the relationship into the entity on the "many" side (the plain line). Inherit the target's primary key as a foreign key.

Resulting Schema:
- `Director(id, name)`
- `Film(title, year, salary, director_id)`

1.8 Ternary Relationships with Mandatory Constraints

The binary "exactly one" optimization **cannot** be applied to ternary relationships. When multiple entities possess plain lines, absorption is impossible.



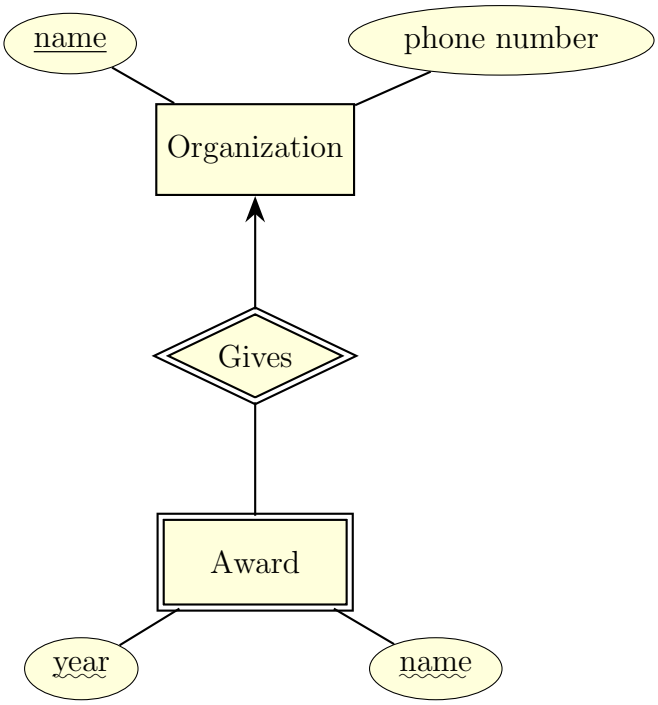
Ternary "Exactly One" Translation

Two entities are unconstrained, meaning a distinct `Studies` table must be generated.
The Rule: The composite primary key is formed from the entities with plain lines. The entity with the rounded arrow contributes a mandatory foreign key (`NOT NULL` constraint).

Resulting Schema:
- `Course(id)`
- `Student(id, name)`
- `Teacher(id, name)`
- `Studies(sid, tid, cid)` *(*cid is NOT NULL)*

1.9 Translating Weak Entity Sets

A **weak entity** lacks sufficient attributes to form its own primary key and depends on a "strong" owner entity via an **identifying relationship**.



Translation Rule: Weak Entity Sets

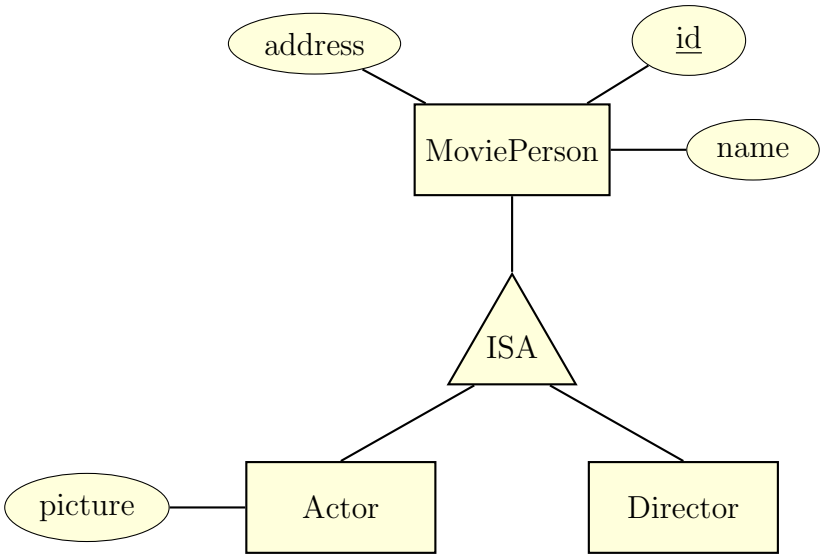
1. **The Weak Entity:** Construct a new relation containing its partial keys *plus* the primary key of its strong owner.
2. **The Primary Key:** The composite of its own partial keys and the owner's primary key.
3. **The Identifying Relationship:** Absorbed via the "Exactly One" optimization.

Resulting Schema: (Attributes renamed to prevent collision).

- `Organization(name, phoneNumber)`
- `Award(aname, year, oname)`

1.10 Translating ISA (Inheritance) Relationships

An **ISA** relationship denotes inheritance. Because relational schemas are inherently non-hierarchical, designers must select one of three storage strategies.



Approach 1: The E/R Style Conversion (Table-per-Class)

A relation is instantiated for *every* entity set. The parent's primary key links the tables.

- `MoviePerson(id, address, name)`
- `Actor(id, picture)`
- `Director(id)`

Pros: Highly normalized. Cons: Requires expensive `JOIN` operations.

Approach 2: The Object-Oriented Approach (Table-per-Concrete-Type)

Relations are created only for valid combinations of entities. Inherited attributes are pushed down.

- `MoviePerson(id, address, name)`
- `MoviePersonActor(id, address, name, picture)`
- `MoviePersonDirector(id, address, name)`
- `MoviePersonActorDirector(id, address, name, picture)`

Pros: Fast read operations. Cons: Difficult schema maintenance and complex data mutation.

Approach 3: The Null Value Approach (Single-Table Strategy)

Collapses the hierarchy into a single, comprehensive relation containing all attributes.

- `MoviePerson(id, address, name, picture, rating)`

Pros: Simple architecture. Cons: Results in sparse tables populated with `NULL` values.

2 Comprehensive Example: Schema Translation

Applying the established translation rules systematically to a complex schema containing strong entities, weak entities, inheritance, and mixed cardinality constraints.

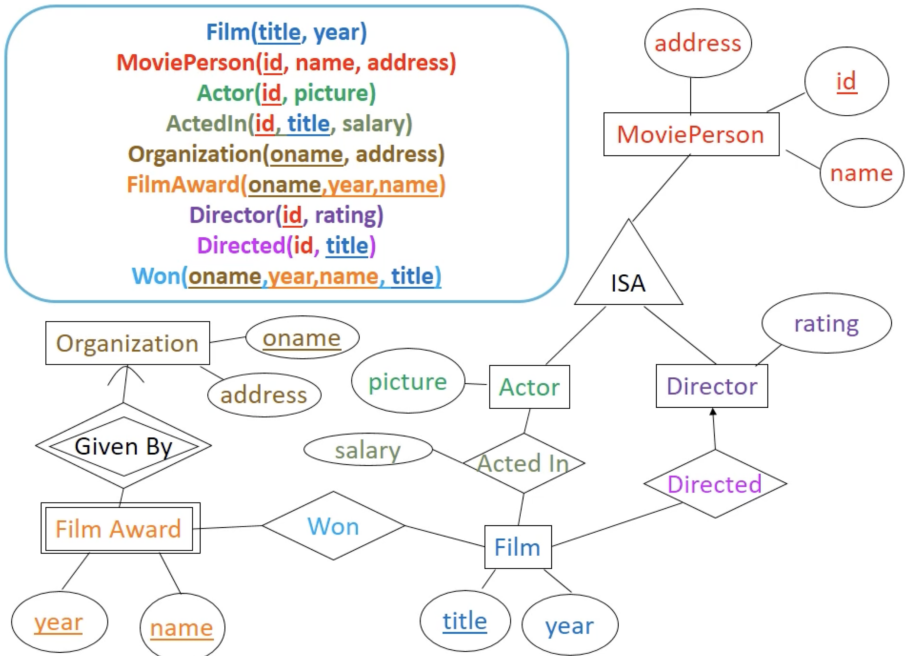


Figure 1: A Complex Example

Step 1: Strong Entities

Translated directly using underlined attributes.

- `Film(title, year)`
- `Organization(oname, address)`

Step 2: ISA Hierarchy (Approach 1)

Using E/R Style conversion.

- `MoviePerson(id, name, address)`
- `Actor(id, picture)`
- `Director(id, rating)`

Step 3: Weak Entities

`Film Award` absorbs `Given By`. Primary key requires the owner's key.

- `FilmAward(oname, year, name)`

Step 4: Many-to-Many Relationships

Require associative tables. Notice `Won` incorporates the composite weak entity key.

- `ActedIn(id, title, salary)`
- `Won(oname, year, name, title)`

Step 5: One-to-Many Relationships

`Directed` is an optional constraint ($\rightarrow$). It requires its own table with the unconstrained side acting as the primary key.

- `Directed(id, title)`